COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE212 - Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	25/08/2026	**DATE SUBMITTED:**	25/08/2026
NAME:	Que, Josef Leroy P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 **Ansible Basics**

1. PROCEDURE OUTPUT

Task 1: Run elevated ad hoc commands

1. Locally, we use the command *sudo apt update* when we want to download package information from all configured resources. The sources are often defined in */etc/apt/sources.list* file and other files located in */etc/apt/sources.list.d/* directory. So, when you run the update command, it downloads the package information from the Internet. It is useful to get info on an updated version of packages or their dependencies. We can only run an apt update command in a remote machine. Issue the following command:

ansible all -m apt -a update_cache=true

```
leroy1@leroyque:~/CPE232_leroyque$ ansible all -i inventory -m apt -a update_cac
he=true

[WARNING]: Host 'node1' is using the discovered Python interpreter at '/usr/bin/
python3.14', but future installation of another Python interpreter could cause a
 different interpreter to be discovered. See https://docs.ansible.com/ansible-co
re/2.20/reference_appendices/interpreter_discovery.html for more information.
[ERROR]: Task failed: Module failed: Failed to lock apt for exclusive operation:
 Failed to lock directory /var/lib/apt/lists/: E:Could not open lock file /var/l
ib/apt/lists/lock - open (13: Permission denied)
Origin: <adhoc 'apt' task>

{'action': 'apt', 'args': {'update_cache': 'true'}, 'timeout': 0, 'async_val': 0
, 'poll': 15}

node1 | FAILED! => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.14"
    },
    "changed": false,
    "msg": "Failed to lock apt for exclusive operation: Failed to lock directory
 /var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open
(13: Permission denied)"
```

What is the result of the command? Is it successful? It will result in failure with permission denied since we need root privileges for updating the cache package.

Try editing the command and add something that would elevate the privilege. Issue the command *ansible all -m apt -a update_cache=true --become --ask-become-pass*. Enter the sudo password when prompted. You will notice now that the output of this command is a success. The *update_cache=true* is the same thing as running sudo apt update. The *--become* command elevates the privileges and the *--ask-become-pass* asks for the password. For now, even if we only changed the packaged index, we were able to change something on the remote server. You may notice after the second command was executed, the status is

CHANGED compared to the first command, which is **FAILED**.

```
leroy1@leroyque:~/CPE232_leroyque$ ansible all -i inventory -m apt -a update_cac
he=true --become --ask-become-pass
BECOME password:
node2 | CHANGED => {
    "cache_update_time": 1787565166,
    "cache_updated": true,
    "changed": true
}
node1 | CHANGED => {
    "cache_update_time": 1787565165,
    "cache_updated": true,
    "changed": true
}
```

2. Let's try to install VIM, which is an almost compatible version of the UNIX editor Vi. To do this, we will just changed the module part in 1.1 instruction. Here is the command: *ansible all -m apt -a name=vim-nox --become --ask-become-pass*. The command would take some time after typing the password because the local machine instructed the remote servers to actually install the package.

```
leroy1@leroyque:~/CPE232_leroyque$ ansible all -i inventory -m apt -a "name=vim-
nox" --become
BECOME password:
node1 | CHANGED => {
    "cache_update_time": 1787565165,
    "cache_updated": false,
    "changed": true,
    "stderr": "",
    "stderr_lines": [],
    "stdout": "Reading package lists...\nBuilding dependency tree...\nReading st
ate information...\nSolving dependencies...\nThe following additional packages w
ill be installed:\n  fonts-lato javascript-common libjs-jquery liblua5.1-0 libru
by libruby3.3\n  libsodium23 rake ruby ruby-csv ruby-did-you-mean ruby-net-telne
t\n  ruby-ruby2-keywords ruby-rubygems ruby-sdbm ruby-webrick ruby-xmlrpc ruby3.
3\n  rubygems-integration vim-runtime\nSuggested packages:\n  apache2 | lighttpd
 | httpd ri ruby-dev bundler cscope vim-doc\nThe following NEW packages will be
installed:\n  fonts-lato javascript-common libjs-jquery liblua5.1-0 libruby libr
uby3.3\n  libsodium23 rake ruby ruby-csv ruby-did-you-mean ruby-net-telnet\n  ru
by-ruby2-keywords ruby-rubygems ruby-sdbm ruby-webrick ruby-xmlrpc ruby3.3\n  ru
bygems-integration vim-nox vim-runtime\n0 upgraded, 21 newly installed, 0 to rem
ove and 138 not upgraded.\nNeed to get 19.7 MB of archives.\nAfter this operatio
n, 90.8 MB of additional disk space will be used.\nGet:1 http://ph.archive.ubunt
u.com/ubuntu resolute/main amd64 fonts-lato all 2.015-1build1 [2779 kB]\nGet:2 h
ttp://ph.archive.ubuntu.com/ubuntu resolute/main amd64 javascript-common all 12+
```

```
leroy1@leroyque:~/CPE232_leroyque$ ansible all -i inventory -m apt -a "name=vim-
nox" --become --ask-become-pass
BECOME password:
node2 | SUCCESS => {
    "cache_update_time": 1787565166,
    "cache_updated": false,
    "changed": false
}
node1 | SUCCESS => {
    "cache_update_time": 1787565165,
    "cache_updated": false,
    "changed": false
}
```

a. Verify that you have installed the package in the remote servers. Issue the command *which vim*

and the command *apt search vim-nox* respectively.

```
leroy1@leroyque:~/CPE232_leroyque$ ansible all -i inventory -a "which vim"
BECOME password:
node2 | CHANGED | rc=0 >>
/usr/bin/vim
node1 | CHANGED | rc=0 >>
/usr/bin/vim
```

```
leroy1@leroyque:~/CPE232_leroyque$ ansible all -i inventory -a "apt search vim-n
ox"
BECOME password:
node2 | CHANGED | rc=0 >>
Sorting...
Full Text Search...
vim-nox/resolute-updates,resolute-security,now 2:9.1.2141-1ubuntu4.8 amd64 [inst
alled]
  Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/resolute-updates,resolute-security,now 2:9.1.2141-1ubuntu4.8 amd64 [ins
talled,automatic]
  Vi IMproved - enhanced vi editor - compact version
WARNING: apt does not have a stable CLI interface. Use with caution in scripts.
node1 | CHANGED | rc=0 >>
Sorting...
Full Text Search...
vim-nox/resolute-updates,resolute-security,now 2:9.1.2141-1ubuntu4.8 amd64 [inst
alled]
  Vi IMproved - enhanced vi editor - with scripting languages support

vim-tiny/resolute-updates,resolute-security,now 2:9.1.2141-1ubuntu4.8 amd64 [ins
talled,automatic]
```

Was the command successful? Yes, it is successful. Which vim returns /usr/bin/vim on the 2 remote servers.

b. Check the logs in the servers using the following commands: *cd /var/log*. After this, issue the command *ls*, go to the folder *apt* and open *history.log*.

```
leroy1@leroyque:~/CPE232_leroyque$ ansible all -i inventory -m command -a "tail
-n 15 /var/log/apt/history.log" --become
BECOME password:
node2 | CHANGED | rc=0 >>
Remove: linux-headers-7.0.0-14:amd64 (7.0.0-14.14), linux-headers-7.0.0-14-gener
ic:amd64 (7.0.0-14.14)
End-Date: 2026-08-24  07:49:52

Start-Date: 2026-08-24  08:07:04
Commandline: apt install -y openssh-server
Requested-By: leroy1 (1000)
Install: ssh-import-id:amd64 (5.11-0ubuntu4, automatic), openssh-server:amd64 (1
:10.2p1-2ubuntu3.5), openssh-sftp-server:amd64 (1:10.2p1-2ubuntu3.5, automatic),
 ncurses-term:amd64 (6.6+20251231-1, automatic)
End-Date: 2026-08-24  08:07:17

Start-Date: 2026-08-24  09:55:08
Commandline: /usr/bin/apt-get -y -o Dpkg::Options::=--force-confdef -o Dpkg::Opt
ions::=--force-confold -o DPkg::Lock::Timeout=60 install vim-nox=2:9.1.2141-1ubu
ntu4.8
Requested-By: leroy1 (1000)
Install: ruby-ruby2-keywords:amd64 (0.0.5-1build1, automatic), ruby-csv:amd64 (3
.3.5-1, automatic), ruby-sdbm:amd64 (1.0.0-5build6, automatic), liblua5.1-0:amd6
4 (5.1.5-12, automatic), fonts-lato:amd64 (2.015-1build1, automatic), libruby:am
```

Describe what you see in the history.log. In this, it outputs the exact timestamp, the requested command which is apt-get install vim-nox, and all of the installed package dependencies.

3. This time, we will install a package called snapd. Snap is pre-installed in Ubuntu system. However, our goal is to create a command that checks for the latest installation package.

   a. Issue the command: *ansible all -m apt -a name=snapd --become --ask-become-pass*

```
leroy1@leroyque:~/CPE232_leroyque$ ansible all -i inventory -m apt -a "name=snap
d" --become --ask-become-pass
BECOME password:
node1 | SUCCESS => {
    "cache_update_time": 1787565165,
    "cache_updated": false,
    "changed": false
}
node2 | SUCCESS => {
    "cache_update_time": 1787565166,
    "cache_updated": false,
    "changed": false
}
```

Can you describe the result of this command? Is it a success? Did it change anything in the remote servers? The command returns success with us seeing the “changed: false” since the snapd is pre-installed on the Ubuntu systems, in which no state change was needed.

b. Now, try to issue this command: *ansible all -m apt -a "name=snapd state=latest" --become --ask-become-pass*

```
leroy1@leroyque:~/CPE232_leroyque$ ansible all -i inventory -m apt -a "name=snap
d state=latest" --become --ask-become-pass
BECOME password:
node2 | CHANGED => {
    "cache_update_time": 1787565166,
    "cache_updated": false,
    "changed": true,
    "stderr": "",
    "stderr_lines": [],
    "stdout": "Reading package lists...\nBuilding dependency tree...\nReading st
ate information...\nSolving dependencies...\nThe following packages will be upgr
aded:\n  snapd\n1 upgraded, 0 newly installed, 0 to remove and 202 not upgraded.
\nNeed to get 42.6 MB of archives.\nAfter this operation, 659 kB of additional d
isk space will be used.\nGet:1 http://ph.archive.ubuntu.com/ubuntu resolute-upda
tes/main amd64 snapd amd64 2.76.3+ubuntu26.04 [42.6 MB]\nFetched 42.6 MB in 22s
(1931 kB/s)\n(Reading database ... \r(Reading database ... 5%\r(Reading database
 ... 10%\r(Reading database ... 15%\r(Reading database ... 20%\r(Reading databas
e ... 25%\r(Reading database ... 30%\r(Reading database ... 35%\r(Reading databa
se ... 40%\r(Reading database ... 45%\r(Reading database ... 50%\r(Reading datab
ase ... 55%\r(Reading database ... 60%\r(Reading database ... 65%\r(Reading data
base ... 70%\r(Reading database ... 75%\r(Reading database ... 80%\r(Reading dat
abase ... 85%\r(Reading database ... 90%\r(Reading database ... 95%\r(Reading da
tabase ... 100%\r(Reading database ... 159791 files and directories currently in
stalled.)\r\nPreparing to unpack .../snapd_2.76.3+ubuntu26.04_amd64.deb ...\r\nU
```

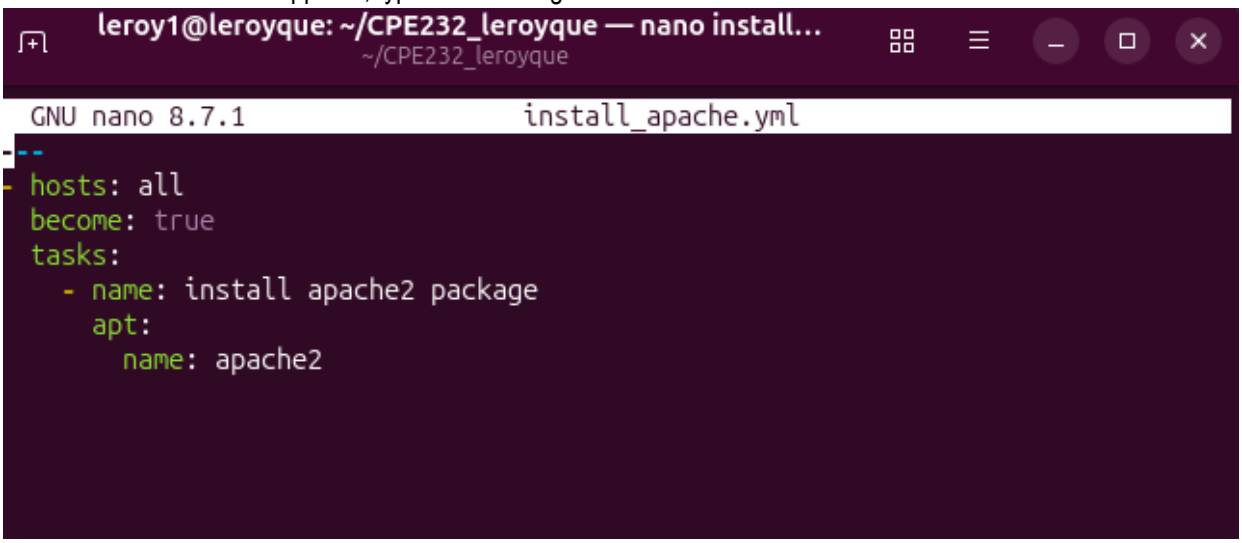
Describe the output of this command. Notice how we added the command state=latest and placed them in double quotations. When we add the state=latest inside the quotes, it will instruct the Ansible to check online repositories and upgrade the snapd to the newest version available release.

Task 2: Writing our First Playbook

1. With ad hoc commands, we can simplify the administration of remote servers. For example, we can install updates, packages, and applications, etc. However, the real strength of ansible comes from its playbooks. When we write a playbook, we can define the state that we want our servers to be in and the place or commands that ansible will carry out to bring to that state. You can use an editor to create a playbook. Before we proceed, make sure that you are in the directory of the repository that we use in the previous activities (CPE232_yourname). Issue the command *nano install_apache.yml*. This will

create a playbook file called *install_apache.yaml*. The .yaml is the basic standard extension for playbook files.

When the editor appears, type the following:



2. Run the yml file using the command: *ansible-playbook --ask-become-pass install_apache.yml.*

```
leroy1@leroyque:~/CPE232_leroyque$ ansible-playbook -i inventory install_apache.
yml
BECOME password:

PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [node1]
ok: [node2]

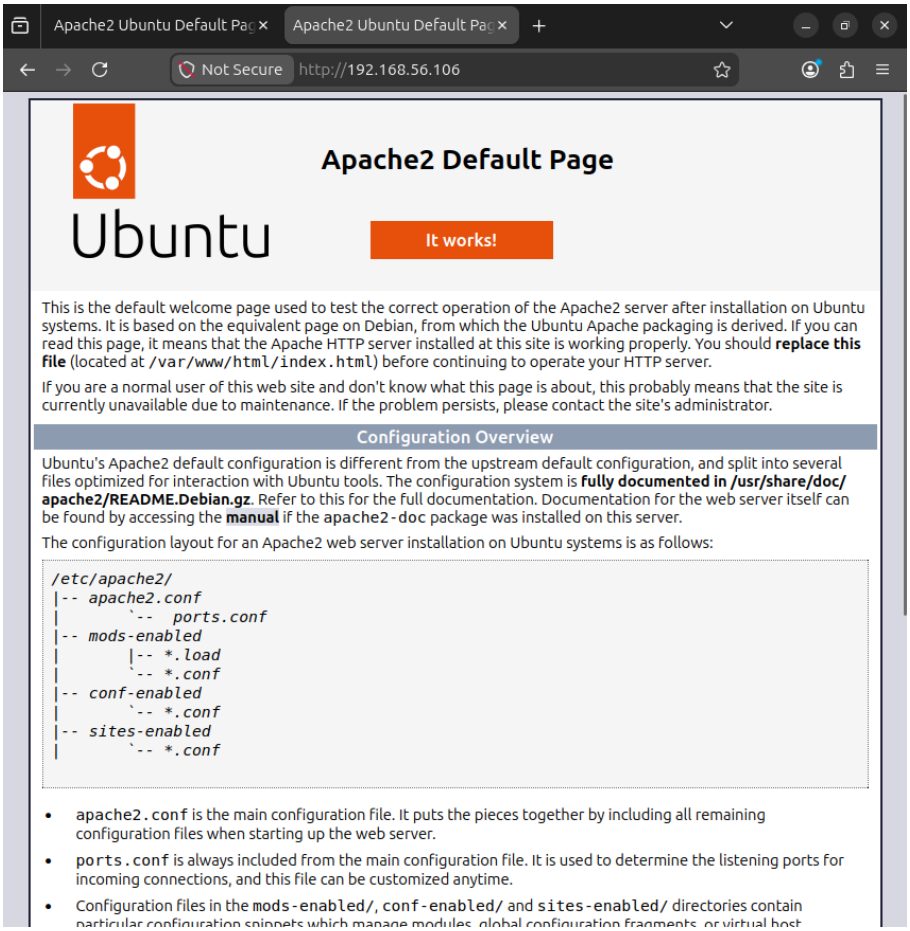
TASK [install apache2 package] *************************************************
changed: [node2]
changed: [node1]

PLAY RECAP *********************************************************************
node1                      : ok=2    changed=1    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
node2                      : ok=2    changed=1    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
```

Describe the result of this command. Here, it executes the playbook across all hosts. The ansible will gather facts, connect through sudo, install the apache2 package on both the nodes, and then return a changed status.

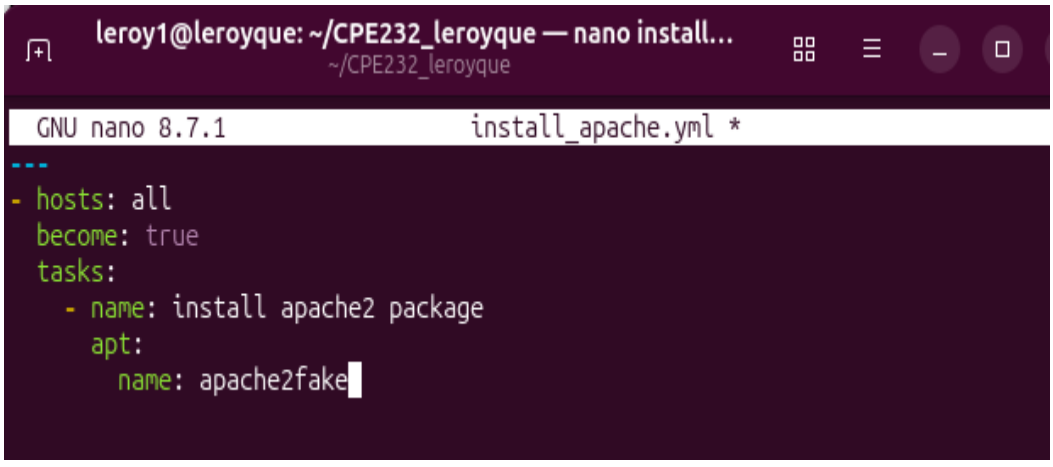
3. To verify that apache2 was installed automatically in the remote servers, go to the web browsers on each server and type its IP address. You should see something like this.





4.

Try to edit the install_apache.yml and change the name of the package to any name that will not be recognized.



```
TASK [install apache2 package] *************************************************
[ERROR]: Task failed: Module failed: No package matching 'apache2fake' is availa
ble
Origin: /home/leroy1/CPE232_leroyque/install_apache.yml:5:7

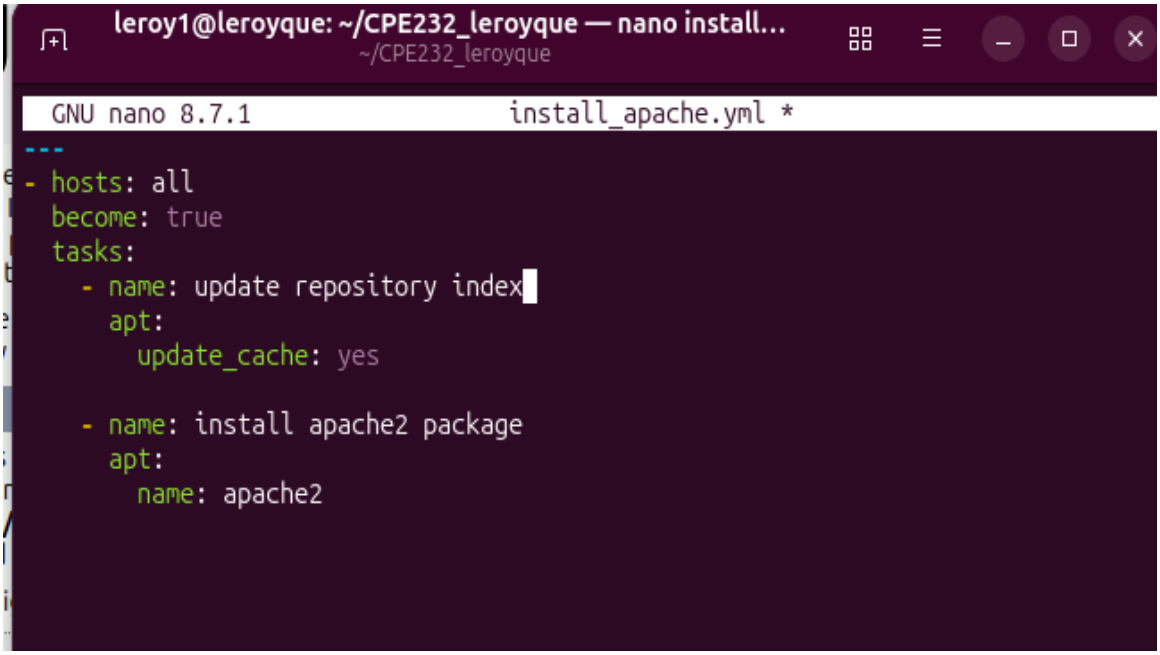
3   become: true
4   tasks:
5     - name: install apache2 package
        ^ column 7

fatal: [node2]: FAILED! => {"changed": false, "msg": "No package matching 'apach
e2fake' is available"}
fatal: [node1]: FAILED! => {"changed": false, "msg": "No package matching 'apach
e2fake' is available"}

PLAY RECAP *********************************************************************
node1                      : ok=1    changed=0    unreachable=0    failed=1    s
kipped=0    rescued=0    ignored=0
node2                      : ok=1    changed=0    unreachable=0    failed=1    s
kipped=0    rescued=0    ignored=0
```

What is the output? It is now unable to locate the package with a fake name. The Ansible halts the execution safely that prevents making invalid changes.

5. This time, we are going to put additional tasks into our playbook. Edit the *install_apache.yml*. As you can see, we are now adding an additional command, which is the *update_cache*. This command updates existing package-indexes on a supporting distro but not upgrading installed-packages (utilities) that were being installed.



Save the changes to this file and exit.

6. Run the playbook and describe the output.

```
leroy1@leroyque:~/CPE232_leroyque$ ansible-playbook -i inventory install_apache.
yml
BECOME password:

PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [node2]
ok: [node1]

TASK [update repository index] *************************************************
changed: [node1]
changed: [node2]

TASK [install apache2 package] *************************************************
ok: [node2]
ok: [node1]

PLAY RECAP *********************************************************************
node1                      : ok=3    changed=1    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
node2                      : ok=3    changed=1    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
```

Did the new command change anything on the remote servers? The update repository index outputs changed which is yellow, while the install apache2 package results ok which is green since Apache is already installed.

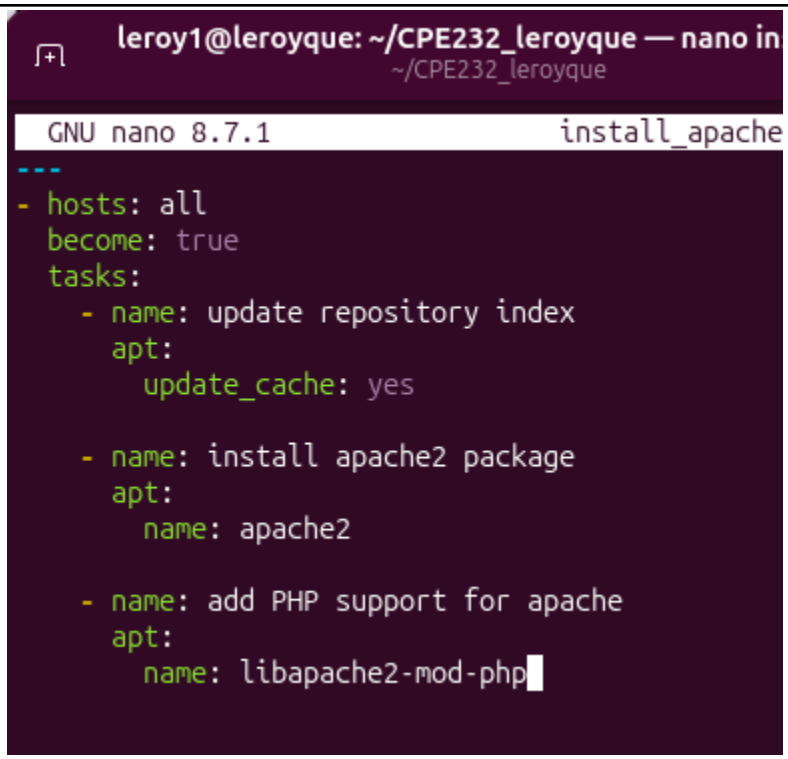
7. Edit again the install_apache.yml. This time, we are going to add a PHP support for the apache package we installed earlier.

```
---
- hosts: all  #change if needed
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes

  - name: install apache2 package
    apt:
      name: apache2

  - name: add PHP support for apache
    apt:
      name: libapache2-mod-php
```



Save the changes to this file and exit.

8. Run the playbook and describe the output. Did the new command change anything on the remote servers?

```
leroy1@leroyque:~/CPE232_leroyque$ ansible-playbook -i inventory install_apache.
yml
BECOME password:

PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [node1]
ok: [node2]

TASK [update repository index] *************************************************
changed: [node2]
changed: [node1]

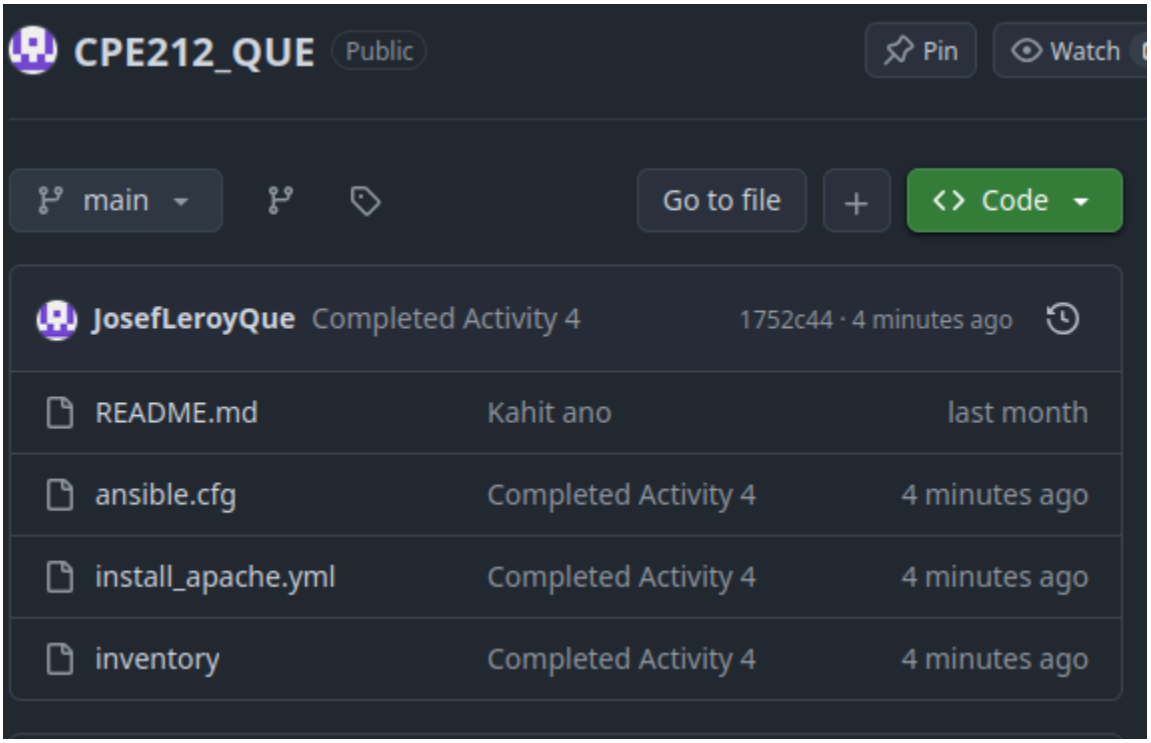
TASK [install apache2 package] *************************************************
ok: [node2]
ok: [node1]

TASK [add PHP support for apache] **********************************************
changed: [node2]
changed: [node1]
```

```
PLAY RECAP *********************************************************************
node1                      : ok=4    changed=2    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
node2                      : ok=4    changed=2    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
```

9. Finally, make sure that we are in sync with GitHub. Provide the link of your GitHub repository.

```
leroy1@leroyque:~/CPE232_leroyque$ git push -u origin main
Username for 'https://github.com': JosefLeroyQue
Password for 'https://JosefLeroyQue@github.com':
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 797 bytes | 797.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/JosefLeroyQue/CPE212_QUE.git
   4a3e529..1752c44  main -> main
branch 'main' set up to track 'origin/main'.
```



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Answer the following:

1. What is the importance of using a playbook?

- Through the use of playbooks, which are both hierarchical and declarative utilizing straightforward YAML syntax, teams can manage to automate intricate IT processes such as the execution of complex multi-stage processes across numerous remote hosts.

2. Summarize what we have done on this activity.

- For this activity, we first acquired root privileges and then access using Ansible ad-hoc commands to manage applications on target machines. After which we created a playbook called install_apache.yml, which we consistently revised to update indexes of repositories and to install the Apache web-server and PHP support on our target machines. At last, we checked the implementation through running a web browser to access the web server and then uploaded our configuration files and playbook into GitHub.

Conclusions/Learnings:

In this activity, I learned about how Ansible plays a crucial role in automating the infrastructure in a very efficient manner by executing its multi-step processes in an orderly fashion across several target hosts. My practical experience involved using elevated privileges correctly in conjunction with apt and doing effective management of the system software, cache repositories, and web applications. The exploration of an intentional failure of a package taught me how Ansible uses its error-handling capabilities to stop a process safely at a certain moment and thus avoid failures in the system. Re-running the playbook showcased the concept of Ansible being an idempotent tool, which means that the processes are being executed only when there is a need for the change. Overall, the combination of Git and GitHub taught me the value of proper versioning while dealing with IAC configurations.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used for explanation of concepts especially for handling errors.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.